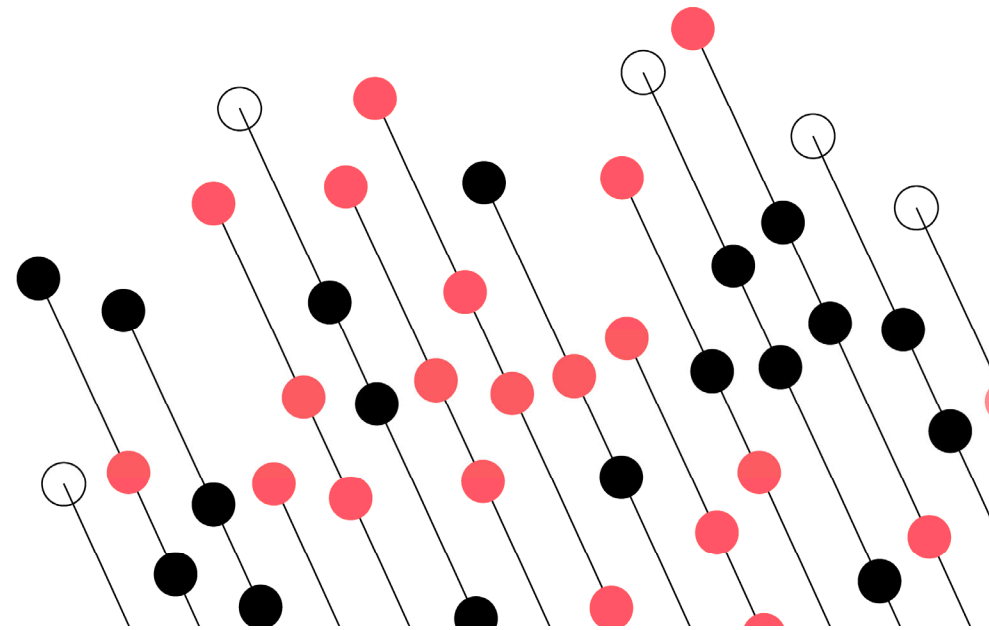


Enum and strings



Contents



Objectives

- Cover some miscellaneous details touched on but not needed so far.



Contents

- Enumerated types — keyword `enum`
- classes `String` and `StringBuilder`
- Review



Hands-on labs

Enumerated data types

- Using the 'class' keyword is not the only way to define a type.
- A variable of the enum is restricted to a set of predefined constants.
 - This is usually defined in uppercase.

```
public enum EmployeeStatus {  
    ACTIVE,  
    RETIRED,  
    CONTRACTOR,  
    PERMANENT;  
}
```

```
EmployeeStatus status = EmployeeStatus.PERMANENT;
```

Note:

- Enum is not a special type of constant, it is a class (reference type).
- Each enum instance is a singleton stored in memory.

Define and use an enum type

```
enum Status
{
    NEW,    IN_PROGRESS,    COMPLETED;
}
```

```
public class Main
{
    public static void main(String[] args)
    {
        Status s1 = Status.COMPLETED;
        System.out.println("Status is: " + s1);
        System.out.println("Status ordinal position is: " + s1.ordinal());
    }
}
```

```
Status is: COMPLETED
Status ordinal position is: 2
```

Enum as method parameter

```
enum Status {  
    NEW,  
    IN_PROGRESS,  
    COMPLETED;  
}
```

```
public static void main(String[] args)  
{  
    var st = Status.NEW;  
    doTask(st);  
}  
  
private static void doTask(Status status)  
{  
    if(status == Status.NEW)  
        // code ...  
}
```

Enum as return value

```
enum Status {  
    NEW,  
    IN_PROGRESS,  
    COMPLETED;  
}
```

```
public static void main(String[] args)  
{  
    Status st = getStatus();  
    // code ...  
}  
  
private static Status doTask()  
{  
    // code ...  
    return Status.COMPLETED;  
}
```

Enum as method parameter — passed by value

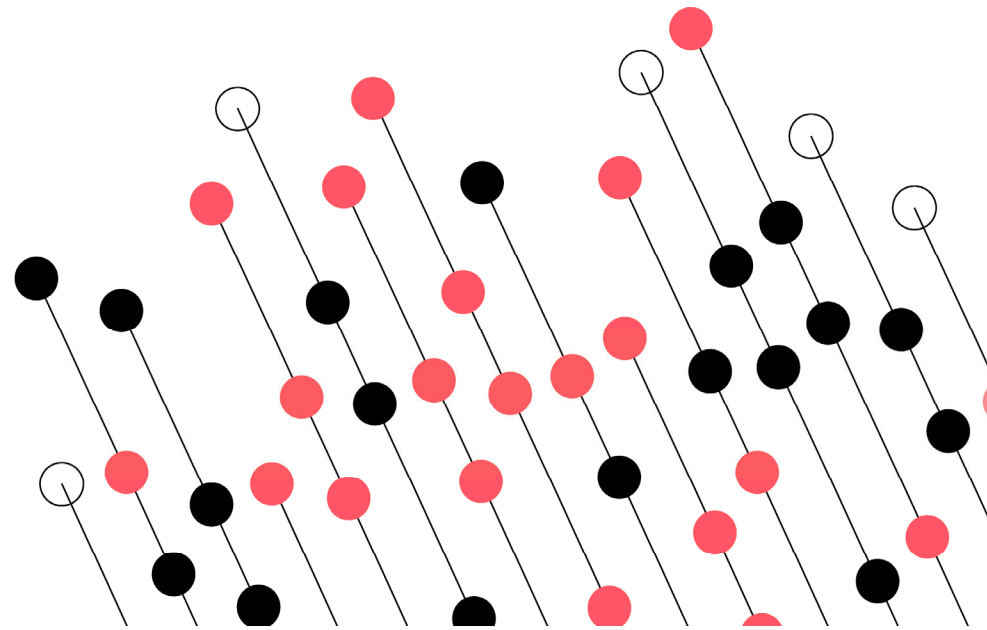
```
enum Status {  
    NEW,  
    IN_PROGRESS,  
    COMPLETED;  
}
```

```
public static void main(String[] args)  
{  
    var x = Status.NEW;  
    changeValue(x);  
    System.out.println(x);  
}  
  
private static void changeValue(Status s)  
{  
    s = Status.COMPLETED;  
}
```

NEW

Enums are immutable and are passed by value

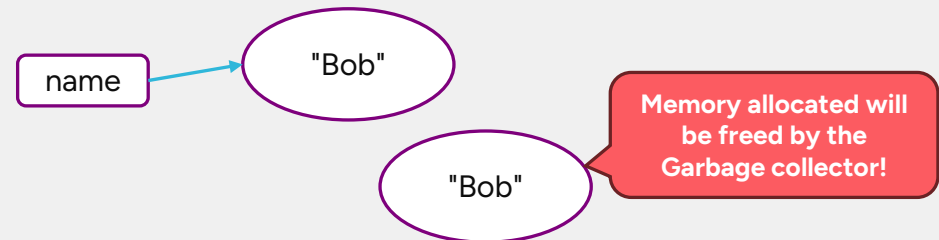
Strings



Strings are reference types and are also immutable

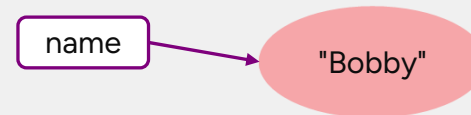
```
String name = "Bob";  
String name = new String("Bob");
```

Is like



```
name = name + "by";  
name = new String(name + "by");
```

Is like

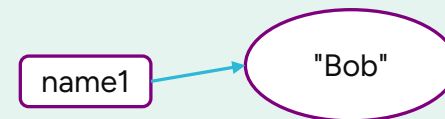


```
name[0] = 'R';
```

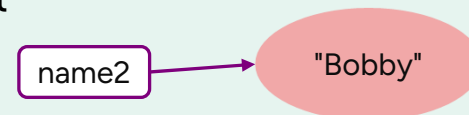


Strings are reference types and are also immutable (2)

```
public static void main(String[] args) {  
    String name1 = "Bob";  
    changeName(name1);  
}
```



```
private static void changeName(String name2) {  
    name2 = "Bobby";  
}
```



name1 remains unchanged

Java: Class string

- String as a class not a primitive, why?

```
public class String {...}
```

It needs to be a class because of its size, but it behaves like a value type.

String objects are 'immutable'.
It has no methods to change its state.

```
String s1 = "Fred";  
String s2 = "Fred";  
System.out.println(s1.equals(s2));
```

True

```
System.out.println(s1 == s2);
```

Indeterminate outcome!

But why?



Java: Class string methods

String has a method `substring`. What will this code display?

```
String s1 = "Frederica";  
System.out.println(s1.substring(2,4));  
  
System.out.println(s1.substring(0, s1.length() - 1));
```

ed
Frederic

And this?

```
String s1 = " Fred";  
s1 += " Bloggs ";  
s1 = s1.trim();  
System.out.println("--" + s1 + "--");  
String s1 = s1.substring(0,4).toLowerCase();  
String s1 = s1.substring(5,s1.length() - 1).toUpperCase();
```

--Fred Bloggs--
fred
BLOGG



Java: Other string methods

```
String s = "Fred Bloggs";  
char c = s.charAt(2);   
for(char c : s.toCharArray()) {...}  
boolean b = s.endsWith("x");  
boolean b = s.startsWith("x");
```

```
int i = s.indexOf('r');  
String[] words= s.split(" ");  
boolean b = s.equalsIgnoreCase(otherString);  
int x = s.length();  
s = s.replace('e','a');
```

None of these methods are void,
you need to catch what they return.

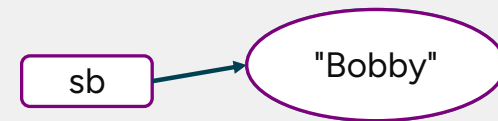
StringBuilder

- **StringBuilder is a mutable String buffer.**
- Key Java methods: `append()`, `insert()`, `replace()`, `delete()`

```
StringBuilder sb = new StringBuilder("Bob");
```

```
sb.append("by");
```

```
String name = sb.toString();
```



Class StringBuilder — examples

```
String s = "Fred";  
for (int i = 0; i < 200; i++) {  
    s += "a";  
}  
print(s);
```



It would display 'Fred' followed by 200 'a's.

But generates 200 string objects to be garbage collected.

- **Any change to a String creates a new String.** An old one will be garbage collected.
 - StringBuilder uses a buffer space in memory.

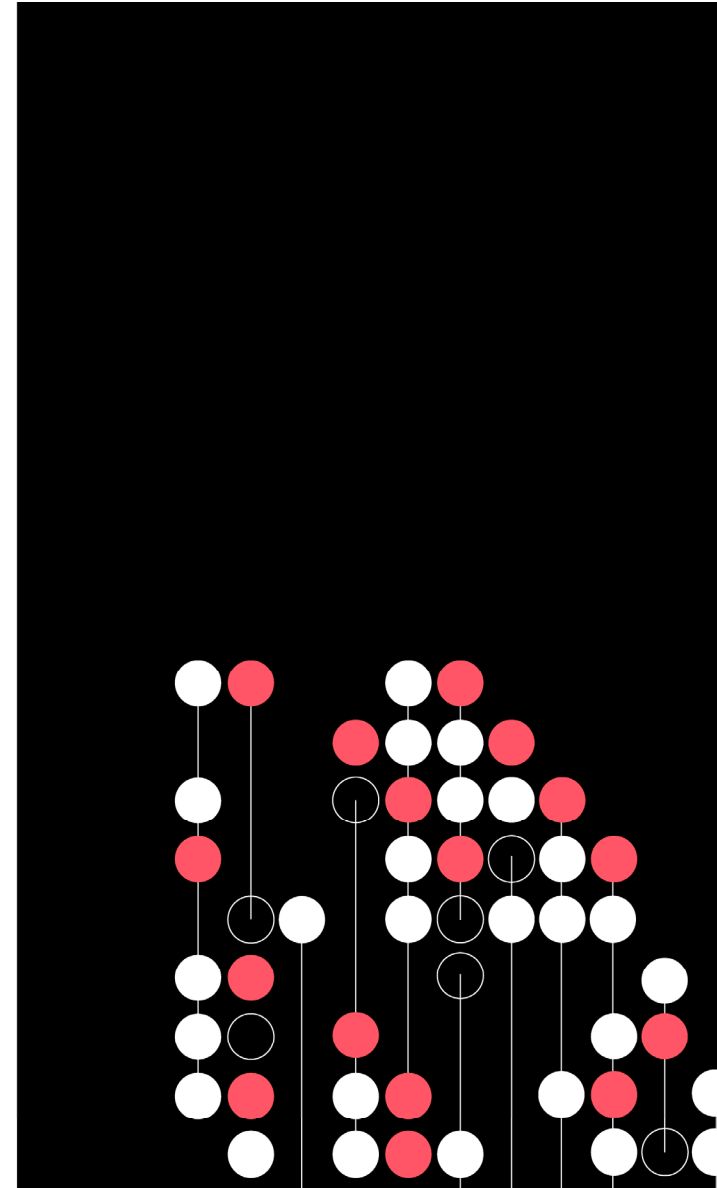
```
StringBuilder sb = new StringBuilder("Fred");  
for(int i = 0; i < 200; i++) {  
    sb.append("a");  
}  
  
print(sb.toString());
```



Fredaaaaaaaaaaaa.....

Review

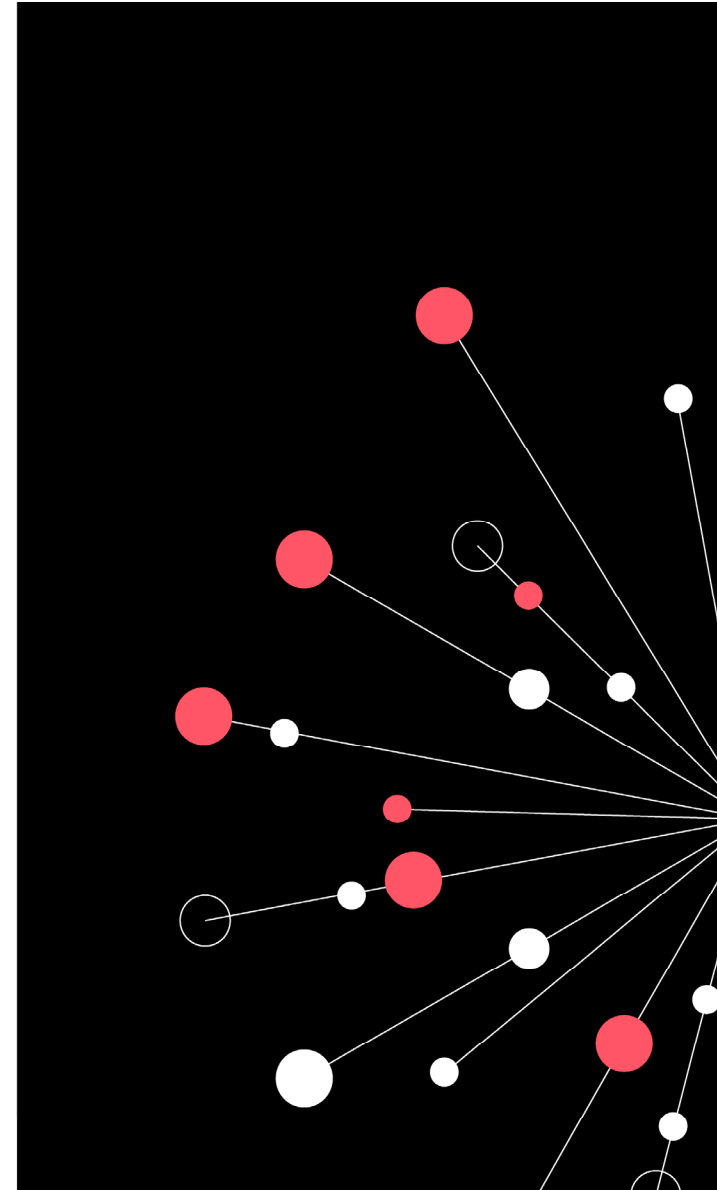
- **Enumerated types — keyword enum**
 - Java creates a class for you. You can store 'extra' data.
- **Classes String and StringBuilder**
 - String is a class but with many value type behaviours.
 - StringBuilder — mutable String Buffer with append, replace, insert.
- **Variable number of parameters — object**
- **Wrapper classes**
 - Integer, double, etc.



Lab

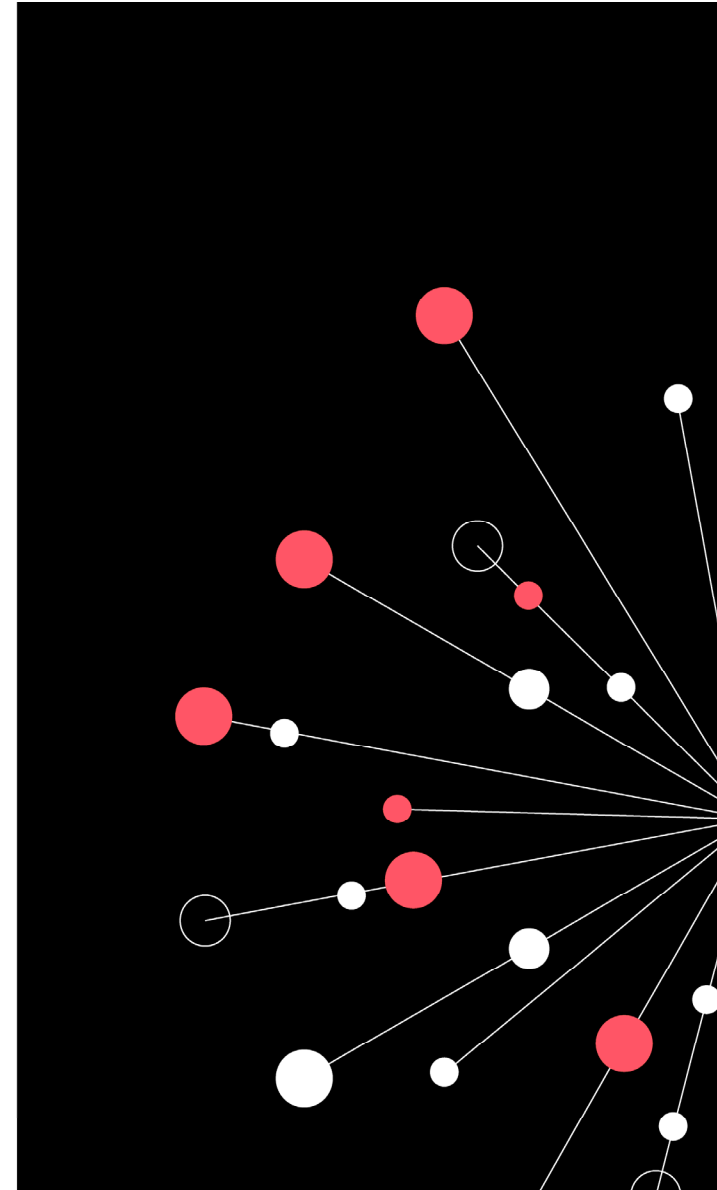
- **Part 1** — Defining and using enum
- **Part 2** — Using String and its key methods
 - Also using class StringBuilder

QA



Extra material

QA



Enum with values

```
enum Status {  
    NEW,  
    IN_PROGRESS,  
    COMPLETED;  
}
```

```
Status myStatus = Status.IN_PROGRESS;
```

```
// Cast String to Status  
myStatus = Status.valueOf("COMPLETED");
```

```
System.out.println("Status value is: " + myStatus);  
System.out.println("Ordinal value is: " + myStatus.ordinal());
```

```
Status value is: COMPLETED  
Ordinal value is: 2
```

```
// throws IllegalArgumentException  
myStatus = Status.valueOf("INVALID");
```

```
Status value is: COMPLETED  
Exception in thread "main" java.lang.IllegalArgumentException: No enum  
constant Status.INVALID
```

Iterating through enum's values

```
enum EmployeeStatus {  
    NEW, IN_PROGRESS, COMPLETED;  
}
```

```
for (EmployeeStatus status : EmployeeStatus.values() )  
{  
    System.out.println(status + ":" + status.ordinal());  
}
```

```
NEW:0  
IN_PROGRESS:1  
COMPLETED:2
```

Assign values

```
enum Status {  
    NEW(100), IN_PROGRESS(20), COMPLETED(30);  
  
    private final int code;  
  
    Status(int code) {  
        this.code = code;  
    }  
    public int getCode() {  
        return code;  
    }  
}
```

```
for (Status s : Status.values() )  
{  
    System.out.println(s + " " + s.getCode());  
}
```

```
NEW:100  
IN_PROGRESS:20  
COMPLETED:30
```